Maintainer Prioritization in npm Supply Chains: Coverage, Exposure, and Graph Representation

Shahriar Hasan*, Ashikul Islam†
*Department of ECE, †Department of CSE
Hajee Mohammad Danesh Science and Technology University

***Abstract*—A maintainer review can cover a downstream repository while leaving other upstream accounts outside the review. We measure this gap in an npm graph containing 25,228 identifiers and 12,217 maintainer usernames; 2,745 identifiers represent bundled contexts rather than separate registry packages. Five selection rules and 200 random orders are evaluated at six budgets. With 100 selected identities, greedy coverage reaches 5,538 of 7,171 workflow-associated repositories (77.2%); exact maintainer-union ranking reaches 4,407. Yet exact union removes 33.79% of account–repository exposure pairs, compared with greedy's 11.48%. This exposure measure assumes a uniformly chosen single-account compromise and perfect hardening of selected accounts. Greedy selects every observed source identity for only 800 repositories. Keeping version identities continuous along paths reduces exposure pairs by 42.44%, with little change in aggregate greedy coverage. The comparison shows why a review program must specify whether it seeks distinct repositories, account-origin exposure reduction, or complete source exclusion. These graph measurements do not establish compromise probabilities or the effectiveness of deployed controls.**

***Index Terms*—software supply chain, npm, maintainer prioritization, dependency graphs, workflow exposure, empirical study**

I. Introduction

An npm repository can depend on packages linked to several maintainer accounts. Earlier studies measured account reach and metadata signals for security review [1], [2]. Selecting accounts for distinct repository coverage can differ from selecting them for exposure across compromise origins.

Recent disclosures show the limits of account selection. SlugFest concerned mutable repository-name trust bindings [3]; GitHub announced expiration of unvalidated bindings on October 2, 2026 [4]. LiteLLM reported master-key exposure, CI secret theft, and developer-token misuse [5], with released artifacts differing from the source tag [6]. Railway described possible environment-variable exposure and a deployment pause [7], [8]. These reports delimit our authority and workflow assumptions; they supply no graph validation.

Reviewing a second account adds little coverage when its reach overlaps the first account's. Leaving it unselected nevertheless retains another compromise origin. We measure both quantities and complete source exclusion, deduplicating packages mapped to one workflow repository.

We ask: **RQ1:** How do coverage and additive exposure selections differ at equal budgets? **RQ2:** How much covered infrastructure remains reachable from unselected identities? **RQ3:** How sensitive are results to representation, endpoint units, and observation gaps?

We compare five rules and 200 random orders at six budgets, then vary representation and observation assumptions. Hybrid and exclusion selections test the tradeoff further. A frozen metadata pilot assesses acquisition feasibility for future outcome research.

II. Related Work

Maintainer prioritization. Our reach sets and coverage baseline build on Zimmermann et al.'s maintainer analysis and vetting proposal [1]. Zahan et al. [2] identify metadata review signals; Wang [9] simulates package-node hardening against cascades. We instead select usernames, removing their account-origin scenarios while retaining other origins' paths. The greedy coverage guarantee is established [10]; exact-union ranking maximizes additive exposure under equal weights and costs.

Dependency semantics. Dependency-tree propagation [11] and version-pinning defenses [12] motivate resolution-aware measurement. Pillar's October 19 OpenCode disclosure concerned Windows/WSL command injection through an older npm `open` dependency, fixed September 28 without a CVE [13]. Shen et al. [14] report 81.26% false positives for package-level findings under finer analysis in their corpus, not npm as a whole. Jayasuriya et al. [15] examine Maven breaking changes. Our version sensitivity checks path continuity without establishing vulnerable-code execution.

Workflow and authority evidence. Studies of workflow security [16], taint analysis in ARGUS [17], and unused CI dependencies [18] examine evidence beyond a directory-presence flag. Signing interviews also document obstacles to putting controls into practice [19]. Work on repository accessibility [20], registry tracking [21], and release-point reconstruction [22] informs our treatment of missing and time-dependent observations. Our contribution is the measured comparison of selection objectives on the retained graph.

III. Data and Graph Construction

A. Collection and Observation Scope

Collection starts from a high-impact npm list and follows dependencies through deps.dev. The collector requests registry metadata and looks for `.github/workflows` at HEAD in linked GitHub repositories (Fig. 1). The retained archive has 25,228 identifiers, 22,340 version-metadata rows, 12,217 usernames, and 79,405 package–maintainer relations. Its queue dates span June 13–16, 2026, but individual fields lack a

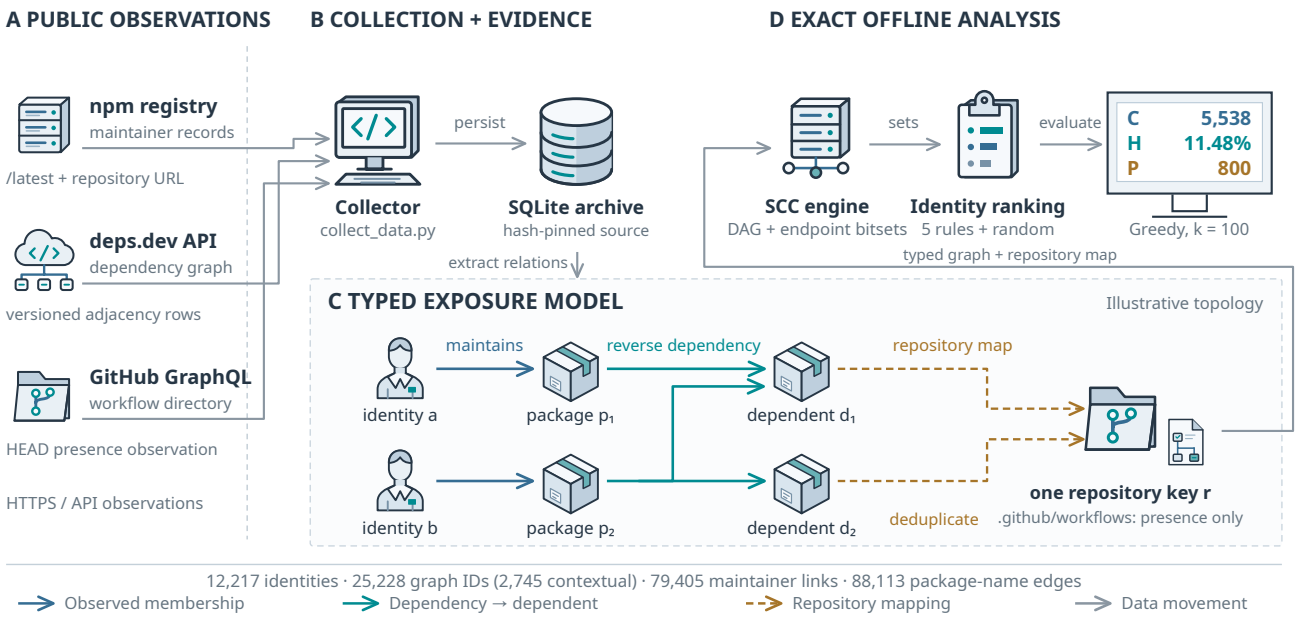


Fig. 1. Measurement architecture. Icons denote logical roles; topology is illustrative. Arrows distinguish membership, reverse dependency reach, and repository mapping. Displayed counts are empirical.

common acquisition cutoff. Our October 20 analysis uses the archive's recorded checksum. Among its identifiers, 2,745 encode bundled contexts under deps.dev's representation [23]. Neither these context nodes nor the seeded expansion constitute a random registry sample.

The data contain 121,885 version-edge rows and 88,113 distinct name adjacencies. We reverse all explicit declaring-to-dependency adjacencies to measure downstream reach. Stored target-node `DIRECT`/`INDIRECT` labels are relative to a resolution root; filtering by them would not recover global direct adjacencies [23].

B. Endpoints and Missing Evidence

We evaluate three endpoint universes: all package names, packages with any positive workflow observation, and distinct repository keys associated with those positive observations. The 15,233 positive packages map to 7,171 repositories. Keys are case-folded owner/name pairs with a trailing transport `.git` removed. Mapping deduplicates repositories without verifying redirects, commits, jobs, or CI installations.

The archive lacks maintainer mappings for 3,081 identifiers and completed workflow flags for 3,772. Moreover, a zero flag can mean either directory absence or repository unavailability. Workflow endpoints therefore require a positive observation. Of the 7,171 such repositories, 7,158 have a reachable maintainer source. We retain 7,171 as the coverage denominator and exclude the 13 source-free repositories from all-source exclusion counts.

C. Two Graph Representations

The primary graph merges versions sharing a package identifier. A second graph retains 40,806 (package, version) nodes and joins paths only at matching version keys. A maintained package seeds all its retained versions; without a version node, it contributes only its eligible seed endpoint. Both graphs include seeds and use the same positive workflow observations. Version continuity removes some paths formed by merging versions. However, retained contexts and acquisition times are still mixed, so neither graph reconstructs a historical installation.

IV. Objectives and Experimental Method

A. Maintainer Exposure Sets

Let M be the observed usernames and P_m the packages linked to m. For an endpoint universe U, we construct

$$E_m = \bigcup_{p \in P_m} \mathcal{D}_U(p), \qquad A_e = \{m \in M : e \in E_m\}, \quad (1)$$

where $\mathcal{D}_U(p)$ contains reverse-reachable endpoints of p, including its eligible seed endpoint. Repository endpoints are deduplicated across every package and path. We condense strongly connected components and propagate exact integer bitsets through the resulting directed acyclic graph. A separate propagation of source-identity bitsets checks the endpoint incidence relation.

B. Three Quantities for the Same Selection

For selected identities $S \subseteq M$, distinct endpoint coverage is

$$C(S) = \left| \bigcup_{m \in S} E_m \right|. \quad (2)$$

For repository endpoints, C counts each repository reached by the selection once, regardless of how many selected accounts reach it.

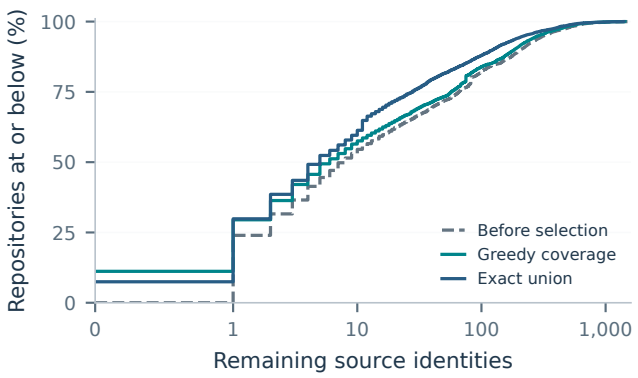


Fig. 2. Remaining sources at budget 100 over 7,158 source-reachable repositories. Zero remaining sources identify all-source exclusions. The horizontal scale is logarithmic above one.

To model source-account hardening, suppose exactly one observed username is selected uniformly as a hypothetical compromise origin. Assume a perfect control prevents that origin when selected, while all other scenarios and graph paths remain unchanged. The proportional reduction in expected endpoint count is then

$$H(S) = \frac{\sum_{m \in S} |E_m|}{\sum_{m \in M} |E_m|}. \tag{3}$$

The uniform prior cancels in the ratio; no compromise probabilities are fitted. Because the numerator adds account-specific reach, sorting by exact $|E_m|$ maximizes H at equal costs. A repository reached by two selected accounts contributes to both scenarios. Selecting one account leaves the other account's paths intact.

Finally, we count endpoints for which every observed source has been selected:

$$P(S) = \sum_{e \in U} \mathbf{1}[A_e \neq \varnothing \ \wedge \ A_e \subseteq S]. \tag{4}$$

We call this *all-source exclusion*. Requiring a nonempty A_e prevents credit for missing source evidence. The difference $C(S) - P(S)$ counts covered endpoints with at least one unselected source. This interpretation depends on the assumed control and observed source sets.

C. Selection Rules and Budget Grid

We compare portfolio size, summed per-package reach, maximum per-package reach, exact portfolio-union reach, and greedy maximum marginal coverage. The sum baseline double-counts overlap within a portfolio. Greedy repeatedly adds the identity contributing the most newly covered endpoints. Coverage is monotone submodular, so this established method has the standard $1 - 1/e$ lower bound relative to optimal coverage [10]; that guarantee does not apply to H or P. The additional exclusion-oriented heuristic has no global optimality guarantee.

Each rule produces nested selections at budgets $1, 5, 10, 25, 50, 100$, with equal account costs and ascending usernames to break ties. The random baseline uses 200 seeded uniform orders without replacement; each order is reused across endpoint universes. Hypergeometric probabilities supply exact expectations from source multiplicities. Central 95% simulation ranges describe variation between selections on this graph. They provide no population confidence interval for npm. We saved an exploratory analysis plan before execution, without external preregistration.

D. Reproducibility and Checks

The read-only Python implementation records hashes, seeds, environment, and definitions. Breadth-first searches independently verify 288 roots and all 25,228 stored package transitive counts; dual incidence, union bounds, nested curves, and $P \leq C$ are checked. Tests cover cycles, version splicing, and 675 roots in 60 seeded small graphs. The grid contains 150 deterministic and 6,000 random budget rows across five endpoint/visibility definitions. No package payload is executed.

Three isolated full reruns reproduce aggregate results byte for byte. On Linux x86-64 with Python 3.14.8, mean wall time is 39.50 s (sample SD 0.98 s), with maximum peak RSS 1,038.4 MiB. These include extraction, baselines, sensitivities, and serialization; uncontrolled caches preclude online-cost inference.

E. Additional Structural Sensitivities

At budget 100, a hybrid greedily maximizes marginal gain of $J_\alpha(S) = \alpha C(S)/|U| + (1-\alpha)H(S)$ at $\alpha = 0.50$. A separate heuristic lexicographically prioritizes immediately completed source sets, fractional remaining-source contributions, marginal coverage, and username order. Candidates come from endpoints with one to ten remaining sources, falling back to all unselected identities.

To distinguish the earlier residual threshold from strict exclusion, define $R_+ = \{e : A_e \neq \varnothing\}$ and

$$T_k(S) = |\{e \in R_+ : |A_e \backslash S| < k\}|, \quad \Delta T_k = T_k(S) - T_k(\varnothing). \tag{5}$$

T_k is a residual-source count, not an enforced review quorum. The strict variant P_k^{strict} counts only endpoints with zero remaining sources or with $|A_e| \geq k$ and fewer than k remaining sources. It has zero baseline; all 13 source-free endpoints are excluded. Uniform efficacy sensitivity $H_\eta = \eta H$ scales the stipulated exposure reduction without estimating a bypass rate.

V. Results

A. RQ1: Coverage and Exposure Favor Different Selections

Table I reports workflow-repository outcomes at 100 identities (0.819% of usernames). Greedy covers 5,538 repositories (77.2%), versus 4,407 (61.5%) for exact union: a descriptive difference of $\Delta C = +1{,}131$ repositories (+25.7%). Exact union instead removes 33.79% of stipulated exposure, versus 11.48% for greedy ($\Delta H = -22.30$ percentage points; $\Delta P = +266$). Its H advantage follows analytically; the experiment quantifies the resulting coverage tradeoff. Maximum reach nears this H but covers fewer repositories.

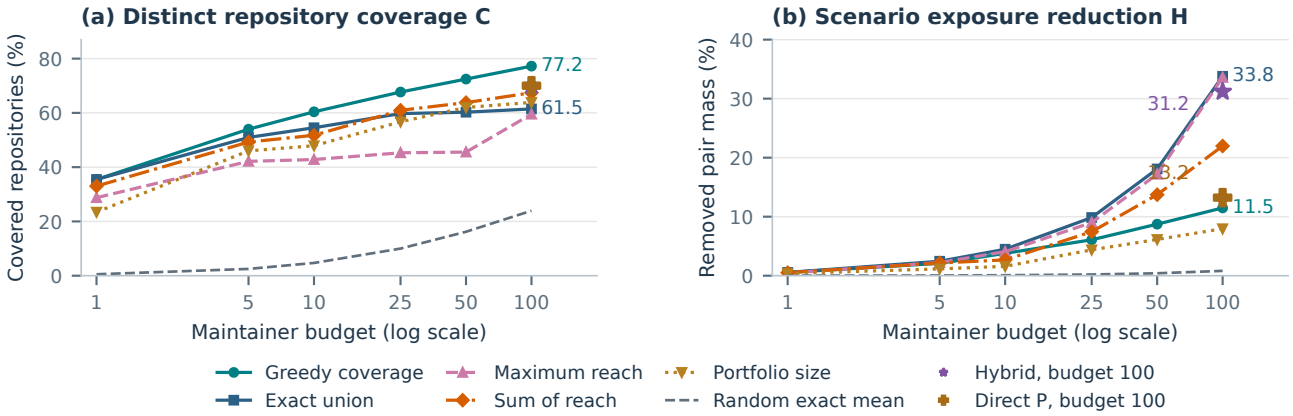


Fig. 3. Workflow-repository coverage and conditional exposure reduction favor different selections. Lines connect the six evaluated budgets. The random curve is the exact expectation; stars and plus markers show the two additional budget-100 sensitivities.

TABLE I
WORKFLOW-REPOSITORY RESULTS AT 100 IDENTITIES. C: DISTINCT COVERAGE; H: CONDITIONAL SCENARIO EXPOSURE REDUCTION; P: ALL-SOURCE EXCLUSION. RANDOM VALUES ARE 200-ORDER MEANS.

Selection	C	C (%)	H (%)	P
Portfolio	4,579	63.9	7.96	577
Summed reach	4,835	67.4	21.98	704
Maximum reach	4,268	59.5	33.59	533
Exact union	4,407	61.5	33.79	534
Greedy coverage	5,538	77.2	11.48	800
Random (mean)	1,680.6	23.4	0.80	12.8

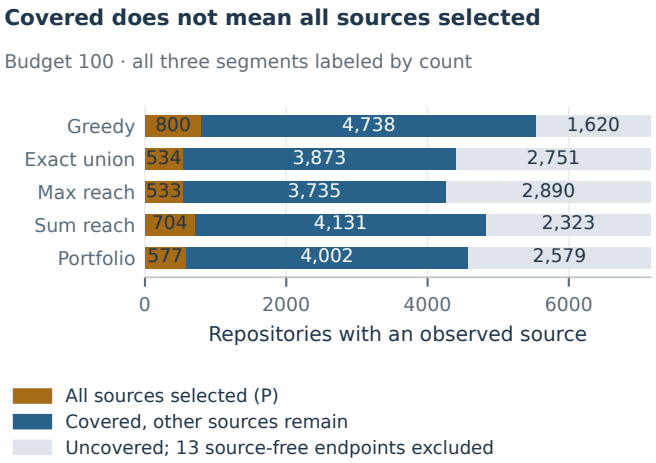


Fig. 4. Repository source-set partition at budget 100. Segment labels give all-source exclusion, covered residual exposure, and uncovered counts. Bars exclude 13 repositories without observed sources. Blue segments remain reachable through unselected identities.

Figure 3 compares all six budgets. At 100, the 200 random orders give mean coverage 1,680.6 (sample SD 530.7; Monte Carlo standard error $\text{MCSE}_C = 37.5$). Their mean H is 0.80% (SD 0.373 pp; $\text{MCSE}_H = 0.026$ pp). The exact expectations are $E[C] = 1{,}716.8$ and $E[H] = 100/12{,}217 = 0.819\%$. These differences arise between finite random selections on the retained graph.

B. RQ2: Covered Endpoints Retain Other Sources

Figure 2 shows source multiplicities; Fig. 4 partitions source-reachable repositories at budget 100. Of the 5,538 repositories covered by greedy, only 800 meet all-source exclusion; 4,738 remain reachable from an unselected identity. Across the 7,158 repositories with nonempty observed source sets, 6,358 retain such a source. Exact-union ranking excludes every observed source for 534 repositories. Neither baseline optimizes P.

Overlap explains why coverage and account-origin exposure diverge. At 100 identities, $1 - C(S)/\sum_{m \in S} |E_m|$ is 89.57% for greedy and 97.18% for exact union. In the retained graph, `qix` is linked to 8 packages and ranks #2,298 by portfolio size. Its 1,942 workflow-repository endpoints place it #7 by exact union; version continuity gives 1,476 and rank #5. Of its name-graph endpoints, 366 lie outside `sindresorhus`'s reach. Greedy selects `sindresorhus` first and `tootallnate` eighth, but omits `qix` from its first 100 selections. Its endpoints therefore retain that unselected source. These are snapshot associations, not historical forecasts.

C. RQ3: Representation Changes Exposure Mass

Preserving version continuity changes downstream closures for 8,587 of 25,228 package roots and removes 1,273,291 ordered root–dependent pairs introduced by package projection. Maintainer workflow-repository unions change for 2,582 usernames. Total identity–repository pairs fall from 462,170 to 266,016, a 42.44% decrease. This is representational sensitivity, not an estimate of false exploit paths in actual installations.

At 100 identities, selecting and evaluating greedy on the version graph covers 5,516 repositories, with $H = 13.10\%$ and $P = 817$. Exact-union ranking gives $C = 4{,}479$, $H = 40.59\%$, and $P = 527$. Evaluating the package-selected greedy identities on that version graph yields $C = 5{,}491$.

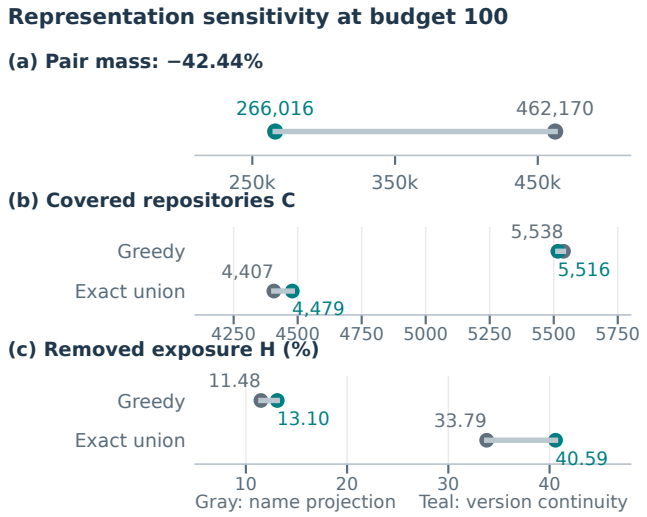


Fig. 5. Version continuity changes pair mass while aggregate coverage remains similar. The lower panels report coverage and exposure, selecting and evaluating within each representation at budget 100. Both graphs remain observed unions.

Aggregate coverage remains similar at this budget, while additive exposure mass is substantially representation-dependent (Fig. 5).

Repository deduplication affects decisions. Selecting greedy for all-package coverage, then evaluating repository endpoints, covers 5,308 repositories; selecting for workflow-package coverage gives 5,315. Direct repository selection gives 5,538. Depth ablation gives 5,255 when selection uses portfolio endpoints alone and 5,488 with one-hop evidence, both evaluated against full reach. Endpoint units and depth therefore matter with the final evaluation denominator held fixed.

VI. Robustness and Research Implications

A. Observation and Selection Sensitivities

The stored repository key disagrees with a lexical URL parse for 60 positive observations. Excluding them removes 24 repository endpoints (7,171 to 7,147); greedy at 100 then gives $C = 5{,}524$ and $P = 799$. We discard those flags instead of assigning them to corrected URLs, which would require fresh observation.

Removing bundled-context nodes preserves every workflow-repository source set and all 462,170 identity–repository pairs. Repository C, H, and P therefore remain unchanged for fixed selections. At the workflow-package level, one endpoint loses 28 incidence pairs; repository deduplication absorbs that change. The invariance is specific to the repository endpoint definition.

Reversing tie order yields $C = 5{,}537$ and $P = 793$. Excluding the 1, 5, or 10 largest portfolios gives coverage of 5,522, 5,149, and 5,076. Grouping equal stored email hashes as a hypothetical control unit yields 12,147 groups with greedy coverage at 5,538. These tests reflect eligibility sensitivity rather than verified credential control.

B. Hybrid, Threshold, and Efficacy Results

The equal-weight hybrid reaches $C = 5{,}020$ (70.0%), $H = 31.19\%$, and $P = 626$ at budget 100. The lexicographic exclusion heuristic reaches the same coverage with $P = 1{,}022$, but H falls to 13.23%. Increasing completed source sets thus has a measurable exposure tradeoff on this graph.

For $k = 2$, $T_2(\varnothing) = 1{,}716$ across 7,158 source-reachable endpoints. Greedy, Exact Union, and Hybrid yield $T_2 = 2{,}112$, 2,137, and 2,153, respectively, giving increments $\Delta T_2 = 396$, 421, and 437. These baseline-adjusted residual counts differ from strict zero-anchored counts: $P_2^{\text{strict}} = 1{,}145$, 793, and 961, respectively. Neither definition measures a deployed multisigner policy.

At assumed efficacy $\eta = 0.90$, Exact Union gives $H_\eta = 30.41\%$ and Hybrid 28.07%, versus Greedy's unattenuated 11.48%. A common multiplier preserves H rankings and does not model correlated CI failures.

C. Prospective Groundwork

Before metadata requests, we froze a uniform sample of 100 graph identifiers without replacement (seed 20261020). Ninety requests returned metadata; eight bundled-context identifiers returned HTTP 405 and two ordinary names returned HTTP 404. The context failures expose a sampling-frame defect. The feature cutoff is October 20, 2026, 00:48:38 Asia/Dhaka, with follow-up ending January 18, 2027. All 100 outcomes are pending. Outcome surveillance, reporting delays, censoring, and family-grouped splits still need specification before analysis; unreported packages remain unlabeled.

VII. Validity, Limitations, and Responsible Reporting

Constructs. Registry membership records a username association; it supplies no token scope, publishing-authority history, or evidence of independent account control. A graph path supplies no execution or secret-access evidence. The scenario prior, efficacy multiplier, and thresholds are assumptions; signer groups and bypass privileges were not observed.

Observation and generalization. Neither representation has a common field cutoff, workflow commits, or complete extraction-root provenance. Mixing resolution contexts can add paths, while missing metadata can omit source identities. The high-impact seed list also prevents treating these measurements as npm-wide estimates.

Internal and conclusion validity. Independent traversals and incidence checks test the implementation against the frozen data. They cannot repair missing observations. The 6,000 random rows reuse orders across nested budgets and endpoint definitions, so they are dependent comparisons rather than independent samples.

Boundaries and reporting. Runtime vulnerabilities such as the Node.js Brotli flaw (CVE-2026-111497) fall outside the npm manifest graph [24]. The LiteLLM artifact mismatch also illustrates why a source tag alone cannot establish a released

build's contents [6]. Our analysis executes no package payloads or credential probes. Prospective captures omit emails and package bodies; no prevention effect was measured.

VIII. Conclusion

The choice of objective changes which maintainers a limited budget selects. Greedy reaches 5,538 workflow repositories at budget 100, but 4,738 retain an unselected source. Exact union sacrifices 1,131 covered repositories while removing a larger share of account-origin exposure. Version continuity reduces pair mass by 42.44%, making graph representation material to that comparison. Hybrid and exclusion heuristics offer further tradeoffs on the same observations. Release-age defenses [12] and Railway's recovery response [8] raise separate operational questions that these experiments do not resolve. Evaluating actual control effects or forecasting compromise requires evidence beyond this graph and the still-pending prospective outcomes.

Acknowledgment

We thank the npm community and the npm registry, deps.dev, and GitHub for public artifacts and interfaces. A large language model (LLM) supported drafting, code revision, analysis checks, and source verification.

References

[1] M. Zimmermann, C.-A. Staicu, C. Tenny, and M. Pradel, "Small world with high risks: A study of security threats in the npm ecosystem," in *28th USENIX Security Symposium*. USENIX Association, 2019, pp. 995–1010. [Online]. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman

[2] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, "What are weak links in the npm supply chain?" arXiv:2112.10165v2, 2022, ICSE SEIP 2022; DOI: 10.1145/3510457.3513044. [Online]. Available: https://arxiv.org/abs/2112.10165v2

[3] Bloom Security, "SlugFest: Millions of AI agents install npm packages exposed to malicious takeover through GitHub," October 2026. [Online]. Available: https://bloom.security/blog/slugfest-millions-of-ai-agents-download-exposed-npm-packages

[4] GitHub, "Unvalidated npm trusted publishing configurations now expire," October 2026. [Online]. Available: https://github.blog/changelog/2026-10-02-unvalidated-npm-trusted-publishing-configurations-now-expire/

[5] O. Jensen and M. Wang, "The attack chain behind the v1.107.0 LiteLLM release," October 2026. [Online]. Available: https://docs.litellm.ai/blog/security-incident-oct-21-2026-attack-chain

[6] LiteLLM Project, "Contained: Oct 21, 2026 LiteLLM release v1.107.0 compromised," October 2026. [Online]. Available: https://docs.litellm.ai/blog/security-incident-oct-21-2026

[7] Railway, "Preliminary report: Oct 21st security incident," October 2026. [Online]. Available: https://blog.railway.com/p/preliminary-report-october-security-incident

[8] J. Cooper, "Security disclosure: Wednesday october 21st," October 2026. [Online]. Available: https://blog.railway.com/p/security-disclosure-wednesday-october-21st

[9] Z. Wang, "Defending the core: A centrality-based protection strategy for supply chain security in npm dependency network," arXiv:2606.14036, 2026, preprint. [Online]. Available: https://arxiv.org/abs/2606.14036

[10] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher, "An analysis of approximations for maximizing submodular set functions—I," *Mathematical Programming*, vol. 14, pp. 265–294, 1978.

[11] C. Liu, S. Chen, L. Fan, B. Chen, Y. Liu, and X. Peng, "Demystifying the vulnerability propagation and its evolution via dependency trees in the NPM ecosystem," in *44th International Conference on Software Engineering*, 2022. [Online]. Available: https://arxiv.org/abs/2201.03981v2

[12] H. He, B. Vasilescu, and C. Kästner, "Pinning is futile: You need more than local dependency versioning to defend against supply chain attacks," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, 2025, article FSE013. [Online]. Available: https://arxiv.org/abs/2502.06662

[13] E. Cohen, "Open and shut: Remote code execution in opencode via MCP OAuth," Pillar Security research disclosure, October 2026. [Online]. Available: https://www.pillar.security/blog/open-and-shut-remote-code-execution-in-opencode-via-mcp-oauth

[14] Y. Shen, X. Gao, H. Sun, and Y. Guo, "Understanding vulnerabilities in software supply chains," *Empirical Software Engineering*, vol. 30, no. 1, p. 20, 2025. [Online]. Available: https://gaoxiang9430.github.io/papers/emse.pdf

[15] D. Jayasuriya, S. Ou, S. Hegde, V. Terragni, J. Dietrich, and K. Blincoe, "An extended study of syntactic breaking changes in the wild," *Empirical Software Engineering*, vol. 30, no. 2, p. 42, 2025.

[16] I. Koishybayev, A. Nahapetyan, R. Zachariah, S. Muralee, B. Reaves, A. Kapravelos, and A. Machiry, "Characterizing the security of GitHub CI workflows," in *31st USENIX Security Symposium*. USENIX Association, 2022, pp. 2747–2763. [Online]. Available: https://www.usenix.org/conference/usenixsecurity22/presentation/koishybayev

[17] S. Muralee, I. Koishybayev, A. Nahapetyan, G. Tystahl, B. Reaves, A. Bianchi, W. Enck, A. Kapravelos, and A. Machiry, "ARGUS: A framework for staged static taint analysis of GitHub workflows and actions," in *32nd USENIX Security Symposium*. USENIX Association, 2023, pp. 6983–7000. [Online]. Available: https://www.usenix.org/conference/usenixsecurity23/presentation/muralee

[18] N. R. Weeraddana, M. Alfadel, and S. McIntosh, "Dependency-induced waste in continuous integration: An empirical study of unused dependencies in the npm ecosystem," in *International Symposium on the Foundations of Software Engineering (FSE)*, 2024, pp. 116:1–116:23. [Online]. Available: https://rebels.cs.uwaterloo.ca/confpaper/2024/07/16/dependency-induced-waste-in-continuous-integration.html

[19] K. G. Kalu, T. Singla, C. Okafor, S. Torres-Arias, and J. C. Davis, "An industry interview study of software signing for supply chain security," in *34th USENIX Security Symposium (USENIX Security 25)*. USENIX Association, 2025, pp. 81–100. [Online]. Available: https://www.usenix.org/conference/usenixsecurity25/presentation/kalu

[20] A. Tsakpinis and A. Pretschner, "Analyzing the accessibility of GitHub repositories for PyPI and NPM libraries," in *28th International Conference on Evaluation and Assessment in Software Engineering*, 2024. [Online]. Available: https://arxiv.org/abs/2404.17403v2

[21] D. Pinckney, F. Cassano, A. Guha, and J. Bell, "npm-follower: A complete dataset tracking the NPM ecosystem," arXiv:2308.12545, 2023. [Online]. Available: https://arxiv.org/abs/2308.12545

[22] I. Rahman, J. Marley, R. Paramitha, and L. Williams, "CHRONO-RESOLUTION: A dependency resolution dataset at release points for npm, PyPI, and crates.io packages," arXiv:2607.15315, 2026, ICSME 2026 Tool Demonstration and Data Showcase Track. [Online]. Available: https://arxiv.org/abs/2607.15315

[23] Open Source Insights, "deps.dev API v3: GetDependencies," Official API documentation, accessed October 20, 2026. [Online]. Available: https://docs.deps.dev/api/v3/

[24] The Node.js Project, "Wednesday, october 21, 2026 security releases," October 2026. [Online]. Available: https://nodejs.org/en/blog/vulnerability/october-2026-security-releases